CONFIGURAR ENTORN DE DESENVOLUPAMENT

Professor: Daniel Jesús García Martínez

Versió: gener 2025

iEbre INSTITUT DE L'EBRE TORTOSA	Departament d'Informàtica	DAM MP10

INTRODUCCIÓ **3**
ARQUITECTURA ODOO **4**
CREAR ENTORN DESENVOLUPAMENT **5**
CREAR UN REPOSITORI PER DESENVOLUPAR ADDONS 5
Crear repositori per addons amb client gh 5
Crear repositori sense client gh 6
CONFIGURAR CONTENIDORS AMB DOCKER COMPOSE 6
Crear configuració 7
Crear repositori per l'entorn de desenvolupament amb client gh 9
AFEGIR REPOSITORI AMB FONTS D'ODOO 10
GESTIONAR LA INSTÀNCIA DOCKER D'ODOO **12**
CREAR CONTENIDORS 12
LLISTAR CONTENIDORS 12
ELIMINAR CONTENIDORS 12
INICIAR CONTENIDORS 12
ATURAR CONTENIDORS 12
REINICIAR CONTENIDORS 13
VOLUMS PER DADES 13
LLISTAR VOLUMS 13
INSPECCIONAR VOLUMS 13
ELIMINAR VOLUMS 14
EXTENSIÓ DOCKER PER VSCODE 14
BASE DE DADES D'EMPRESA **15**
CREAR UNA NOVA BASE DE DADES D'EMPRESA 15
ACTIVAR EL MODE DEVELOPER 16
IMPORTAR BASE DE DADES D'EMPRESA 16
TREBALLAR AMB DOCKER **20**
DOCKER SENSE SUDO 20
CONSULTAR LOGS D'UN CONTENIDOR 20
INSTAL·LACIÓ DEL CLIENT POSTGRESQL **21**
INSTAL·LACIÓ 21
Configurar repositori APT 21
Instal·lar amb apt la versió desktop 21
Obrir pgadmin4 22
CONFIGURACIÓ 23

INTRODUCCIÓ

El desenvolupament en Odoo el realitzarem fent servir contenidors Docker. D'aquesta manera partirem d'una versió estable i operativa d'Odoo (18.0) a més d'estalviar recursos evitant l'ús de màquines virtuals.

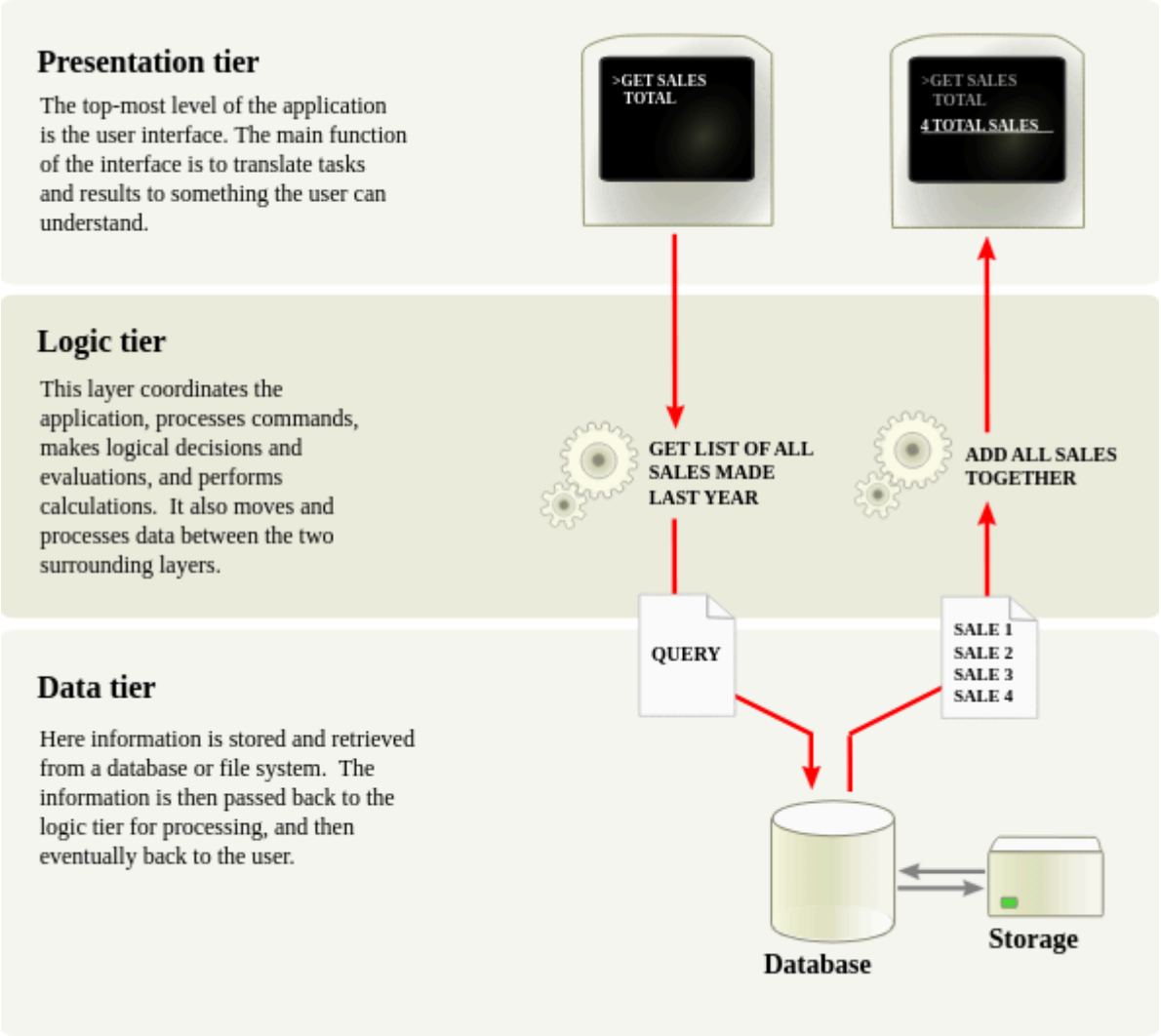
Com documentació de referència disposem de la documentació de la imatge oficial d'Odoo:

https://hub.docker.com/_/odoo/

ARQUITECTURA ODOO

Per conèixer l'arquitectura d'Odoo és important llegir i comprendre la introducció que fan en el seu tutorial per desenvolupadors:

https://www.odoo.com/documentation/18.0/developer/tutorials/server_framework_101/01_architecture.html



Presentació: combinació de HTML5, Javascript i CSS.
Lògica: Python
Dades: PostgreSQL

CREAR ENTORN DESENVOLUPAMENT

La guia principal és el tutorial d'Odoo fent les adaptacions i personalitzacions necessàries.

En el nostre cas, farem servir Docker en lloc d'instal·lar un servidor com màquina virtual o configurar servidor i base de dades en local.

https://www.odoo.com/documentation/18.0/developer/tutorials/setup_guide.html

CREAR UN REPOSITORI PER DESENVOLUPAR ADDONS

Com afegirem el codi dels addons desenvolupats a un repositori propi de Git, el primer pas serà crear-lo. Treballarem en GitHub i, per tant, s'ha de disposar de compte. Si fos necessari crear un compte es pot seguir la guia:

https://www.odoo.com/documentation/18.0/contributing/development.html

Tot i que el tutorial parteix del fet que farem contribucions a les versions Community i Enterprise d'Odoo, obviarem aquesta part per anar directament a desenvolupar addons en un repositori independent de creació pròpia. El tutorial ens proporciona un repositori de base a partir del qual crear el nostre propi, però pel nostre curs partirem d'un entorn repositori buit.

En la nostra màquina creem un directori que contindrà tot el nostre entorn de desenvolupament Odoo. Per aquest tutorial el podem anomenar "mp10-uf2-odoo". Des del "home" del nostre usuari:

```
mkdir mp10-uf2-odoo
cd mp10-uf2-odoo
```

Crear repositori per addons amb client gh

Seguidament, inicialitzem el repositori on desenvoluparem els addons:

```
mkdir odoo-tutorials

cd odoo-tutorials

git init -b main
```

```
git add .
git commit -m "First commit"
```

A continuació pugem el repositori local a GitHub amb el seu client si el tenim instal·lat:

```
~/mp10-uf2-odoo/odoo-tutorials$ gh repo create
```

Crear repositori sense client gh

Sense client gh:

```
git init -b main
git add .
git commit -m "First commit"
```

Tot seguit cal autenticar-se en GitHub des del terminal (https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github#authenticating-with-the-command-line)

Des del lloc GitHub, crearem un nou repositori i obtindrem l'url.

Finalitzarem vinculant el repositori local amb el remot (reemplaçar REMOTE-URL amb l'url obtingut prèviament):

```
git remote add origin REMOTE-URL
git push origin main
```

CONFIGURAR CONTENIDORS AMB DOCKER COMPOSE

Per facilitar la gestió de múltiples contenidors i disposar d'una configuració fàcilment reutilitzable, disposem de l'eina "docker compose". En un fitxer de configuració podem definir tots els contenidors amb els seus paràmetres. Així, podrem recrear els contenidors quan sigui necessari, a més de disposar de comandes per iniciar, aturar i reiniciar el conjunt de contenidors vinculats.

Crear configuració

Dins de “mp10-uf2-odoo” definim un directori “odoo-dev” que servirà per crear un fitxer docker-compose.yml.

```
cd ~/mp10-uf2-odoo
mkdir odoo-dev
cd odoo-dev
nano docker-compose.yml
```

En l’editor indiquem:

- La versió de la imatge d’Odoo (18.0).
- Els diferents serveis
- Paràmetres específics de la imatge (per exemple, paràmetres per desenvolupadors). En concret, - - dev all permet carregar vistes de l’aplicació directament des de fitxers i no des de base de dades (evitant haver d’actualitzar l’aplicació a cada canvi).
- Les dependències d’altres contenidors
- El mappeig de ports
- Els volums (data, configuració d’odoo i addons)
- Paràmetres d’entorn com passwords, usuaris i rutes.
- Definició de “secrets”, per externalitzar passwords d’aquest fitxer.

El contingut pel nostre cas és:

```
services:
  web:
    image: odoo:18.0

    command: odoo --update all --dev all

    depends_on:
      - db
    ports:
```

```
      - "8069:8069"
    volumes:
      - odoo-web-data:/var/lib/odoo
      - ./config:/etc/odoo
      - ./addons:/mnt/extra-addons
    environment:
      - PASSWORD_FILE=/run/secrets/postgresql_password
    secrets:
      - postgresql_password
  db:
    image: postgres:15
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_PASSWORD_FILE=/run/secrets/postgresql_password
      - POSTGRES_USER=odoo
      - PGDATA=/var/lib/postgresql/data/pgdata
    ports:
      - "5432:5432"
    volumes:
      - odoo-db-data:/var/lib/postgresql/data/pgdata
    secrets:
      - postgresql_password
volumes:
  odoo-web-data:
  odoo-db-data:

secrets:
  postgresql_password:
    file: odoo_pg_pass
```

En aquest cas, s'ha externalitzat el password a un fitxer anomenat odoo_pg_pass en el mateix directori. Aquest mecanisme és útil per desvincular informacions delicades de fitxers públics així com per a gestionar diferents entorns (test, integració, preproducció, producció), on l'única diferència és el valor del password.

Editem un fitxer odoo_pg_pass i afegim el valor del password.

En el fitxer de configuració d'Odoo definit "./config/odoo.conf" s'ha d'afegir la ruta als addons. Es tracta d'una configuració interna del contenidor (tot i que la modifiquem des de l'amfitrió gràcies al "binding" fet) i, per tant, s'han d'indicar rutes internes del contenidor. Editem el fitxer:

```
mkdir config

nano config/odoo.conf
```

Afegim:

```
[options]
addons_path = /mnt/extra-addons
```

Per últim, lliguem el directori “addons” (el crearem dins “odoo-dev”) al directori on realment desenvoluparem els addons. Ho fem amb un “soft link”:

```
ln -s ../odoo-tutorials addons
```

El directori “odoo-dev” hauria de contenir:

```
~/mp10-uf2-odoo/odoo-dev$ ls -l | awk '{print $1, $10, $11, $12}'
total
lrwxrwxrwx addons -> ../odoo-tutorials
drwxr-xr-x config
-rw-rw-r-- docker-compose.yml
-rw-rw-r-- odoo_pg_pass
```

Crear repositori per l’entorn de desenvolupament amb client gh

Finalment, convertim aquest directori en un repositori de git:

```
~/mp10-uf2-odoo/odoo-dev$ git init -b main

git add .
git commit -m "First commit"
```

A continuació pugem el repositori local a GitHub amb el seu client si el tenim instal·lat:

```
~/mp10-uf2-odoo/odoo-dev$ gh repo create
? What would you like to do? Push an existing local repository to GitHub
? Path to local repository .
? Repository name odoo-dev
? Repository owner danieljesusgarcia
? Description
? Visibility Private

✓ Created repository danieljesusgarcia/odoo-dev on GitHub

? Add a remote? Yes

? What should the new remote be called? origin

✓ Added remote git@github.com:danieljesusgarcia/odoo-dev.git

? Would you like to push commits from the current branch to "origin"? Yes

Enumerating objects: 14, done.

Counting objects: 100% (14/14), done.

Delta compression using up to 4 threads

Compressing objects: 100% (11/11), done.

Writing objects: 100% (14/14), 2.82 KiB | 413.00 KiB/s, done.

Total 14 (delta 4), reused 0 (delta 0), pack-reused 0

remote: Resolving deltas: 100% (4/4), done.

To github.com:danieljesusgarcia/odoo-dev.git

 * [new branch]      HEAD -> main

Branch 'main' set up to track remote branch 'main' from 'origin'.

✓ Pushed commits to git@github.com:danieljesusgarcia/odoo-dev.git
```

AFEGIR REPOSITORI AMB FONTS D'ODOO

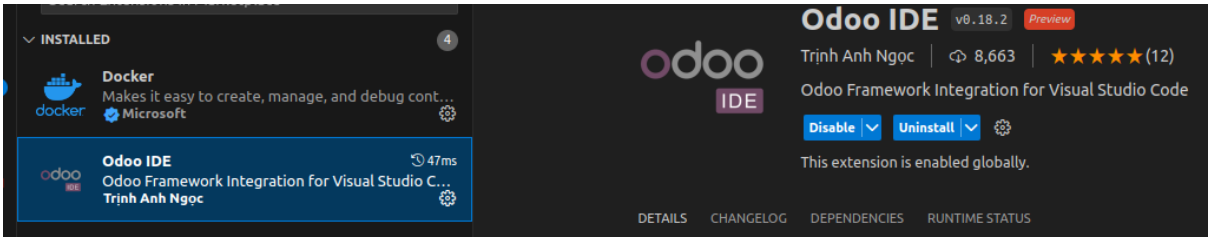
Per desenvolupar més fàcilment i poder reconèixer els mòduls d'Odoo que fem servir al nostre codi (autocompleció, intellisense), farem un "clone" del repositori oficial d'Odoo amb la mateixa versió que la imatge. El fem sota "mp10-uf2-odoo":

```
cd ~/mp10-uf2-odoo
```

Seguidament, fem el “clone” del repositori d’Odoo:

```
git clone -b 18.0 --single-branch --depth 1 git@github.com:odoo/odoo.git
```

Per reconèixer els mòduls d’Odoo dins de qualsevol nou creat, instal·larem també l’extensió ODOO IDE:



GESTIONAR LA INSTÀNCIA DOCKER D'ODOO

CREAR CONTENIDORS

Una vegada ja tenim definit l'entorn de desenvolupament amb "docker compose" i hem creat també un directori pels addons, iniciem l'aplicació des del directori "odoo-dev" creant per primera vegada els contenidors:

```
cd ~/mp10-uf2-odoo/odoo-dev

docker compose up -d
```

LLISTAR CONTENIDORS

Per llistar els contenidors existents:

```
docker ps -a
```

ELIMINAR CONTENIDORS

Si és necessari eliminar els contenidors ho podem fer amb:

```
docker compose down
```

Els contenidors s'eliminaran, però es conservaran els volums definits com el de la base de dades.

INICIAR CONTENIDORS

Per iniciar els contenidors ja creats:

```
docker compose start
```

ATURAR CONTENIDORS

Per aturar els contenidors en execució:

```
docker compose stop
```

REINICIAR CONTENIDORS

Per reiniciar els contenidors ja creats i en execució (stop i start):

```
docker compose restart
```

VOLUMS PER DADES

Els volums permeten conservar i compartir dades més enllà del contenidor. Per exemple, el volum es conservarà tot i haver eliminat el contenidor.

LLISTAR VOLUMS

Si creem un contenidor amb volums, aquests no s'esborren en suprimir el contenidor. Per conèixer els volums existents:

```
docker volume list
```

INSPECCIONAR VOLUMS

Per inspeccionar els detalls de cada volum executem (on "my-vol" és el nom del volum):

```
docker volume inspect my-vol
```

Per odoo-data obtenim:

```
docker volume inspect odoo-dev_odoo-web-data

[
    {
        "CreatedAt": "2025-01-13T13:32:30+01:00",
        "Driver": "local",
```

```
        "Labels": {
            "com.docker.compose.project": "odoo-dev",
            "com.docker.compose.version": "2.21.0",
            "com.docker.compose.volume": "odoo-web-data"
        },
        "Mountpoint": "/var/lib/docker/volumes/odoo-dev_odoo-web-data/_data",
        "Name": "odoo-dev_odoo-web-data",
        "Options": null,
        "Scope": "local"
    }
]
```

Per tant, el directori de l'amfitrió on es munta el volum és:

```
/var/lib/docker/volumes/odoo-data/_data
```

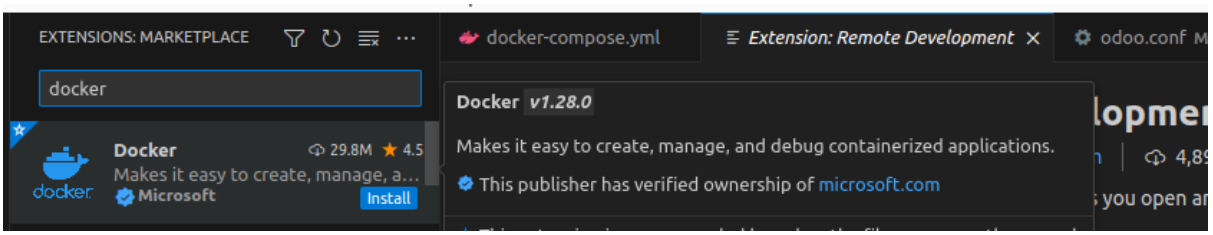
ELIMINAR VOLUMS

Per eliminar un volum que pugui resultar erroni o per reinicialitzar les dades vinculades al contenidor, els podem eliminar amb:

```
docker volume rm <nom_volum>
```

EXTENSIÓ DOCKER PER VSCODE

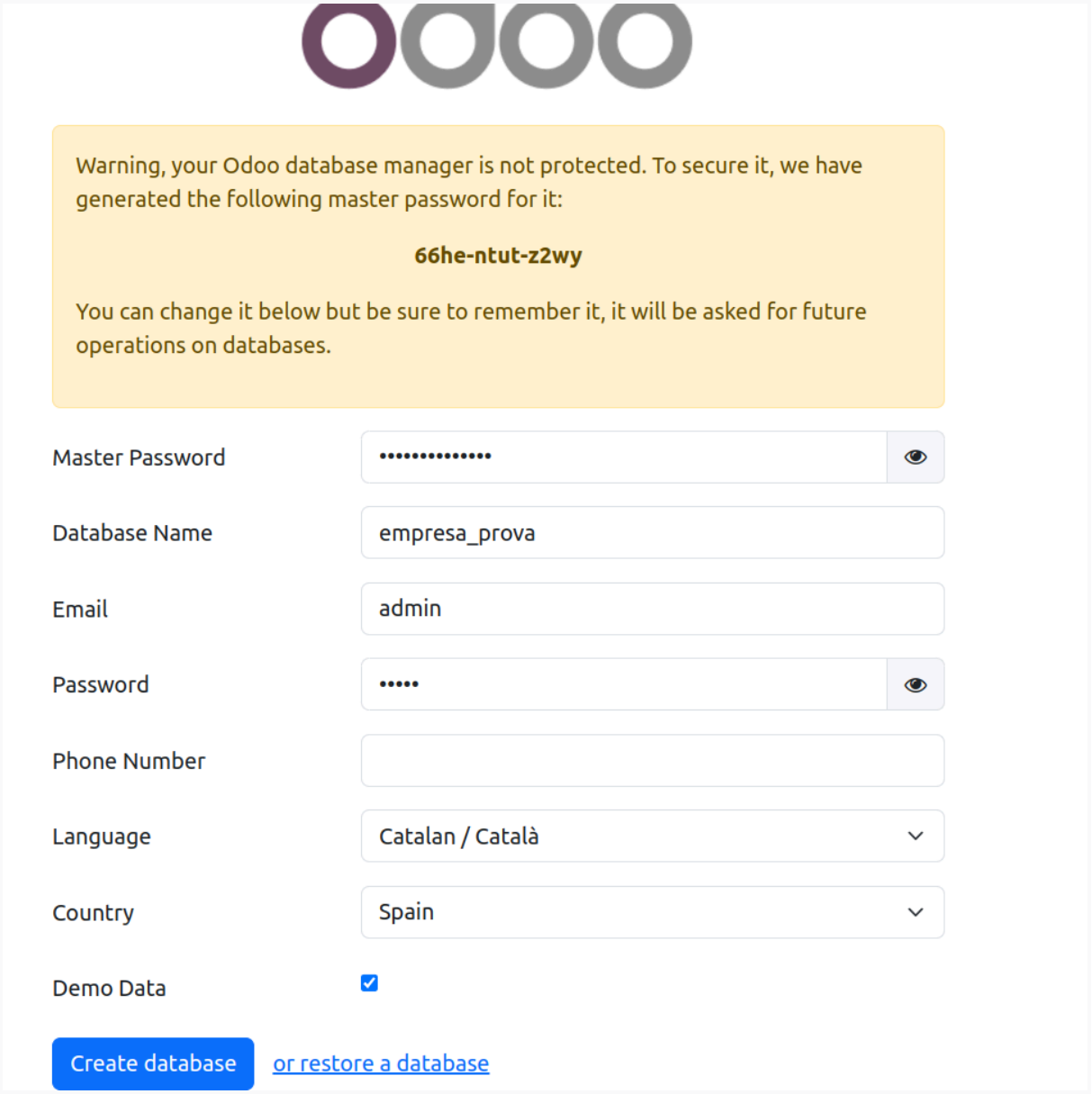
L'extensió de Docker per VSCode ens permetrà gestionar els contenidors directament des de l'IDE. La instal·lem:



BASE DE DADES D'EMPRESA

CREAR UNA NOVA BASE DE DADES D'EMPRESA

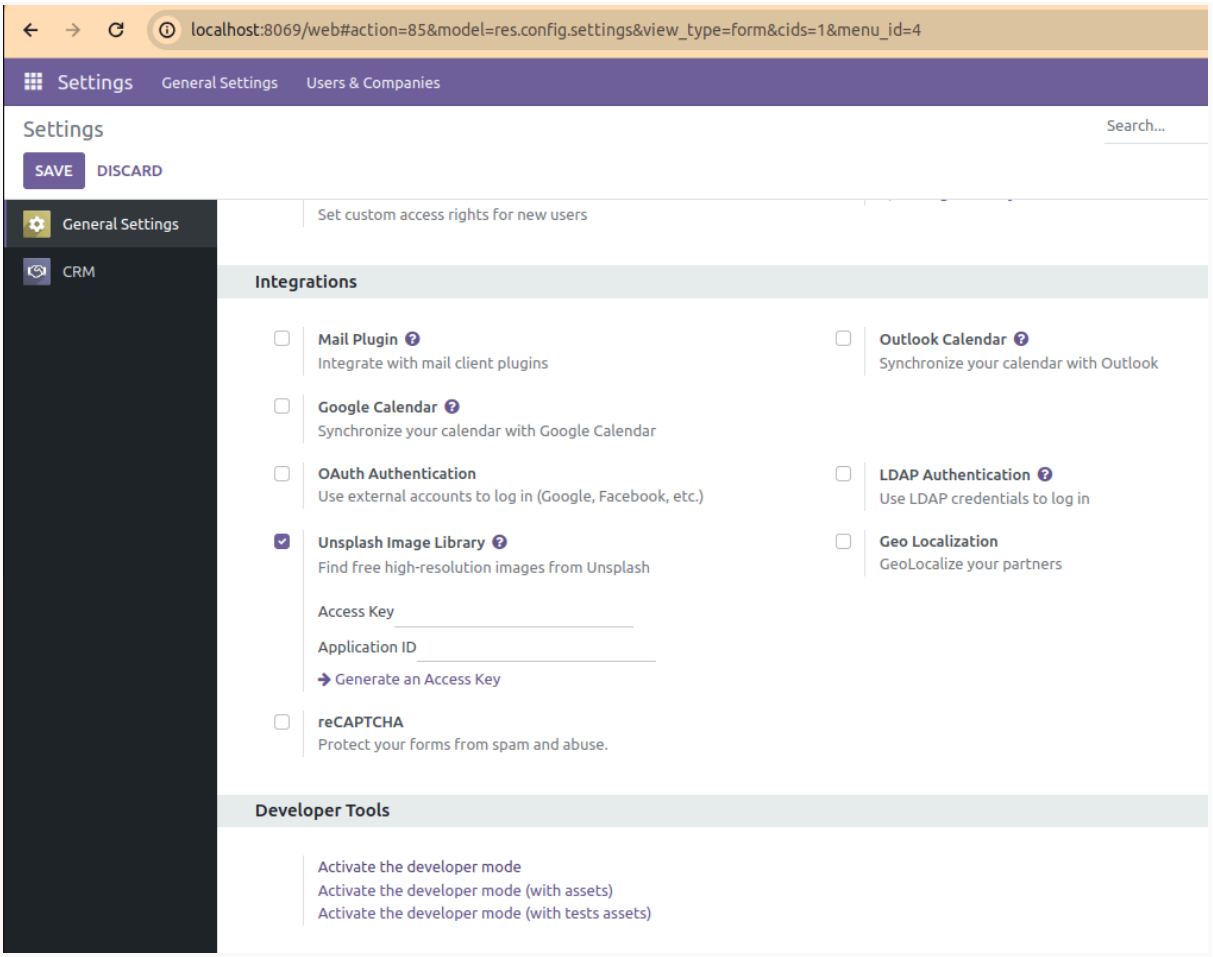
Seguidament, ens connectem a l'url http://localhost:8069 per crear una base de dades d'empresa:



ACTIVAR EL MODE DEVELOPER

Per disposar d'informació i eines addicionals per desenvolupadors es pot activar el mode desenvolupador. Abans cal activar una app (p.e. CRM) per poder accedir a la configuració completa:

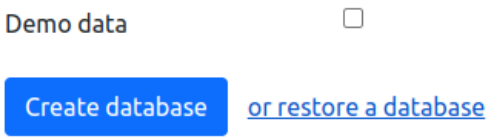
https://www.odoo.com/documentation/master/applications/general/developer_mode.html#developer-mode



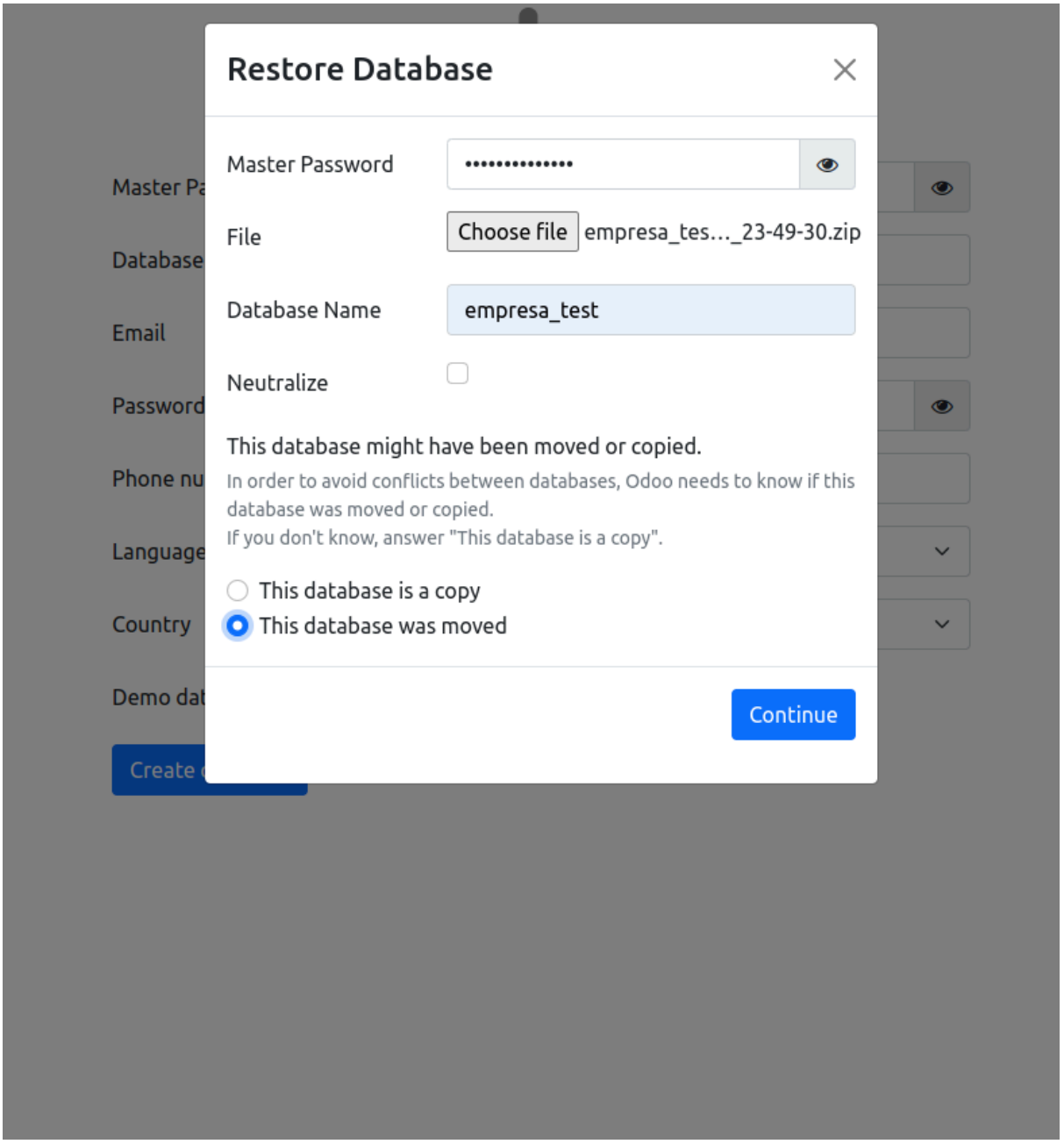
IMPORTAR BASE DE DADES D'EMPRESA

Si ja disposem d'una base de dades d'empresa creada, podem restaurar-la (prèviament hauríem d'haver fet una exportació en la instància original).

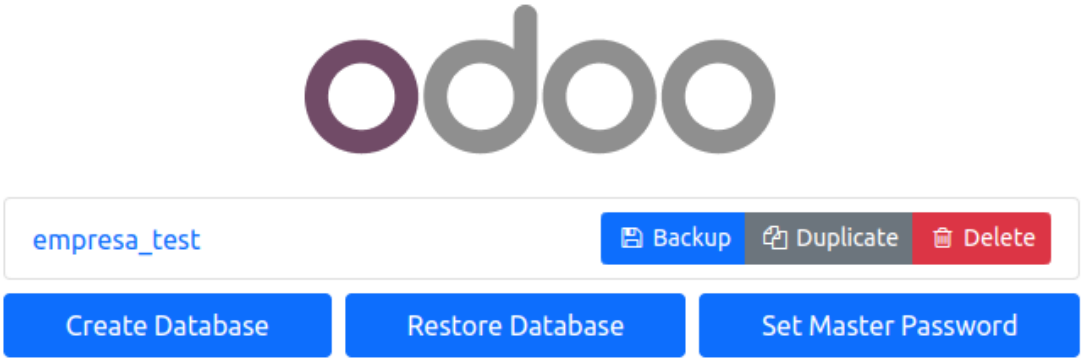
Cliquem sobre l'opció "restore a database":



I seleccionem el fitxer de la base de dades existent. Si no existeix la base de dades en aquesta instància seleccionem que es mou la base de dades:



Com a resultat, disposem de la base de dades importada:


odoo
empresa_test
Backup
Duplicate
Delete
Create Database
Restore Database
Set Master Password

TREBALLAR AMB DOCKER

DOCKER SENSE SUDO

Com la comanda “docker” es fa servir constantment i per defecte només un usuari “root” la pot executar, podem afegir el nostre usuari al grup d’usuaris de “docker” per tal d’autoritzar-lo a executar les diferents comandes. El detall es pot trobar en:

https://docs.docker.com/engine/install/linux-postinstall/

CONSULTAR LOGS D’UN CONTENIDOR

Els logs d’Odoo es generen per defecte per sterr. Per passar-lo per un “pipe”, fem servir “|&”. I per redirigir-los a un fitxer, “&>”

Per exemple, per obtenir les 50 últimes línies del contenidor odoo-dev-web-1 i aplicar un filtre “grep”:

```
docker logs odoo-dev-web-1 --tail 50 |& grep -n empresa_test
```

Per obtenir les 50 últimes línies del contenidor odoo-dev-web-1 i redirigir-les a un fitxer:

```
docker logs odoo-dev-web-1 --tail 50 &> logfile.txt
```

Per veure els logs des d’una línia:

```
docker logs odoo-dev-web-1 |& tail +3980 | more
```

Per veure els logs en viu a mesura que es van generant:

```
docker logs -f odoo-dev-web-1
```

INSTAL·LACIÓ DEL CLIENT POSTGRESQL

Podem accedir a la instància del gestor de base de dades mitjançant el terminal i l'eina "psql" sense dependre de cap altre programari, especialment en un entorn no gràfic. Però un client gràfic ens facilita molt la tasca. Tot i que hi ha altres clients de base de dades, l'específica per PostgreSQL és pgAdmin.

INSTAL·LACIÓ

A continuació instal·larem pgAdmin4 (la guia completa es troba a https://www.pgadmin.org/download/pgadmin-4-apt/)

Configurar repositori APT

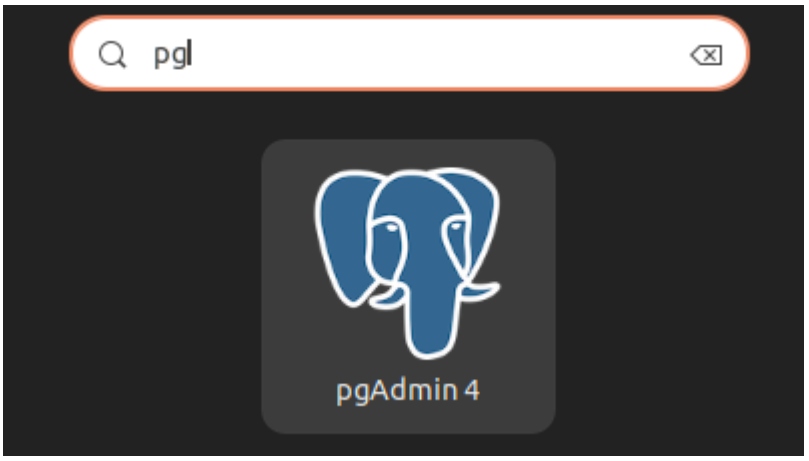
```
curl -fsS https://www.pgadmin.org/static/packages_pgadmin_org.pub | sudo gpg
--dearmor -o /usr/share/keyrings/packages-pgadmin-org.gpg
```

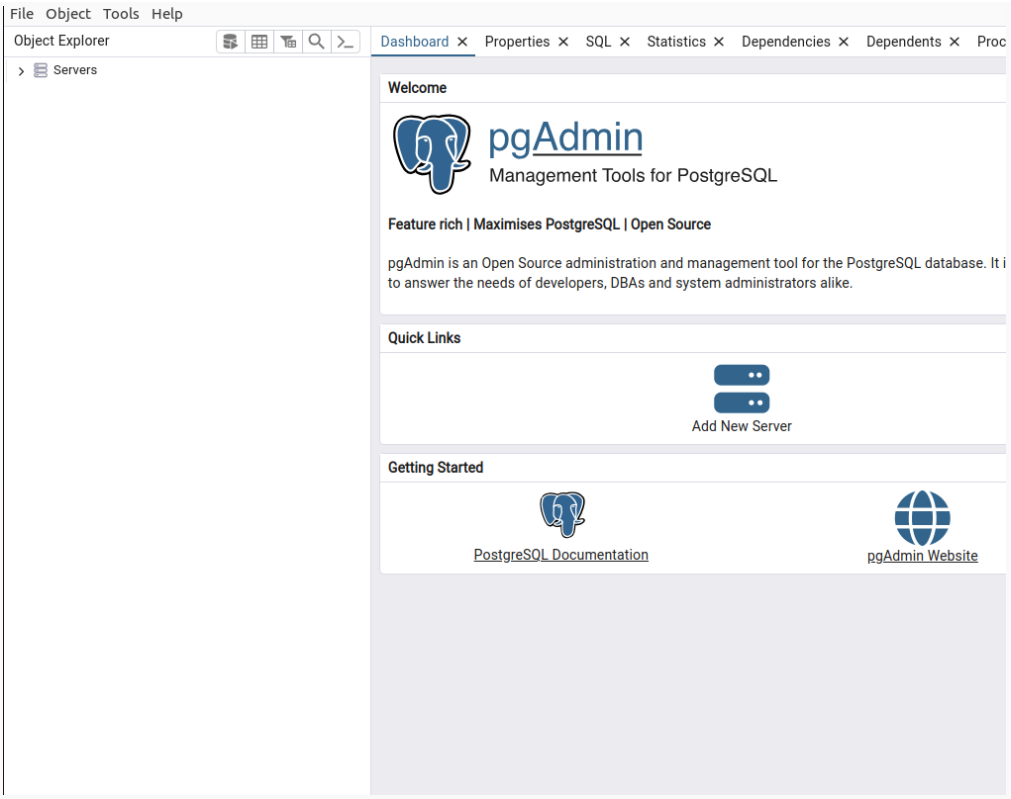
```
sudo sh -c 'echo "deb
[signed-by=/usr/share/keyrings/packages-pgadmin-org.gpg]
https://ftp.postgresql.org/pub/pgadmin/pgadmin4/apt/$(lsb_release -cs)
pgadmin4 main" > /etc/apt/sources.list.d/pgadmin4.list && apt update'
```

Instal·lar amb apt la versió desktop

```
sudo apt install pgadmin4-desktop
```

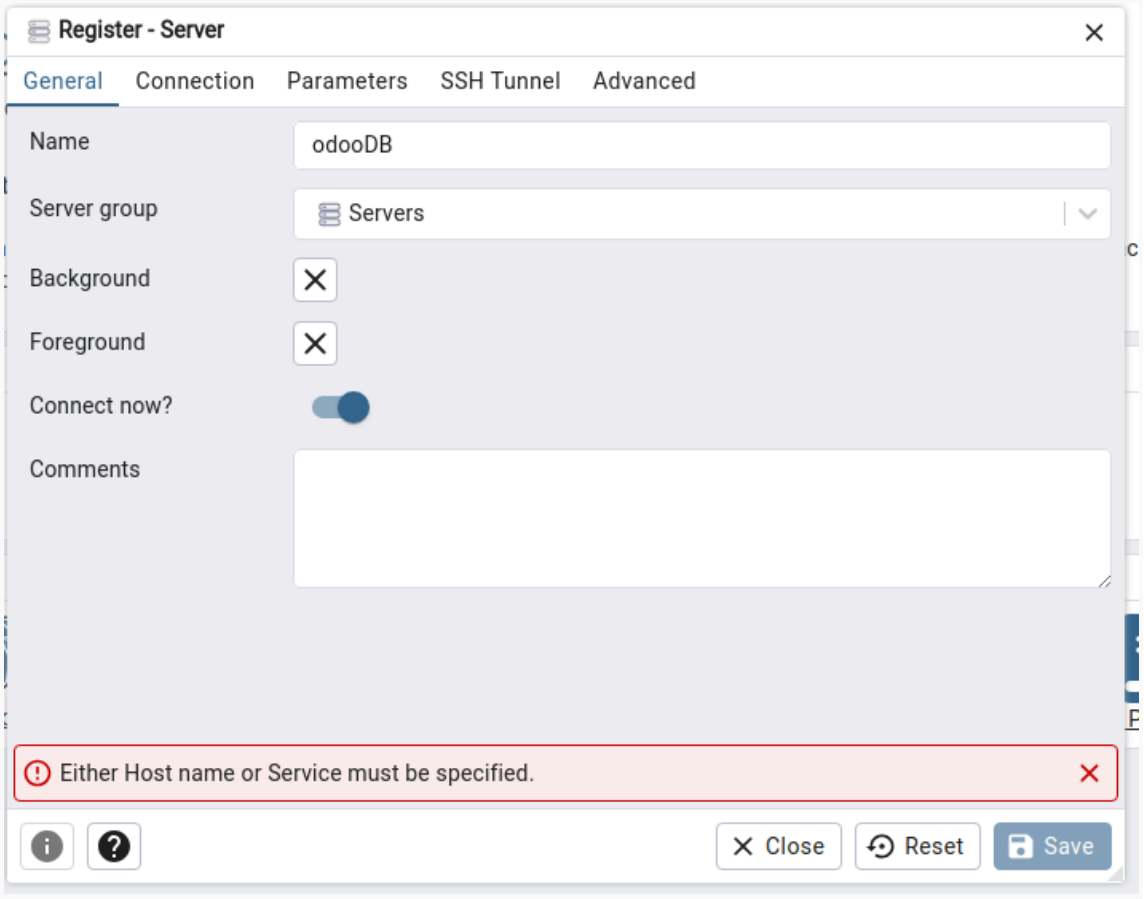
Obrir pgadmin4



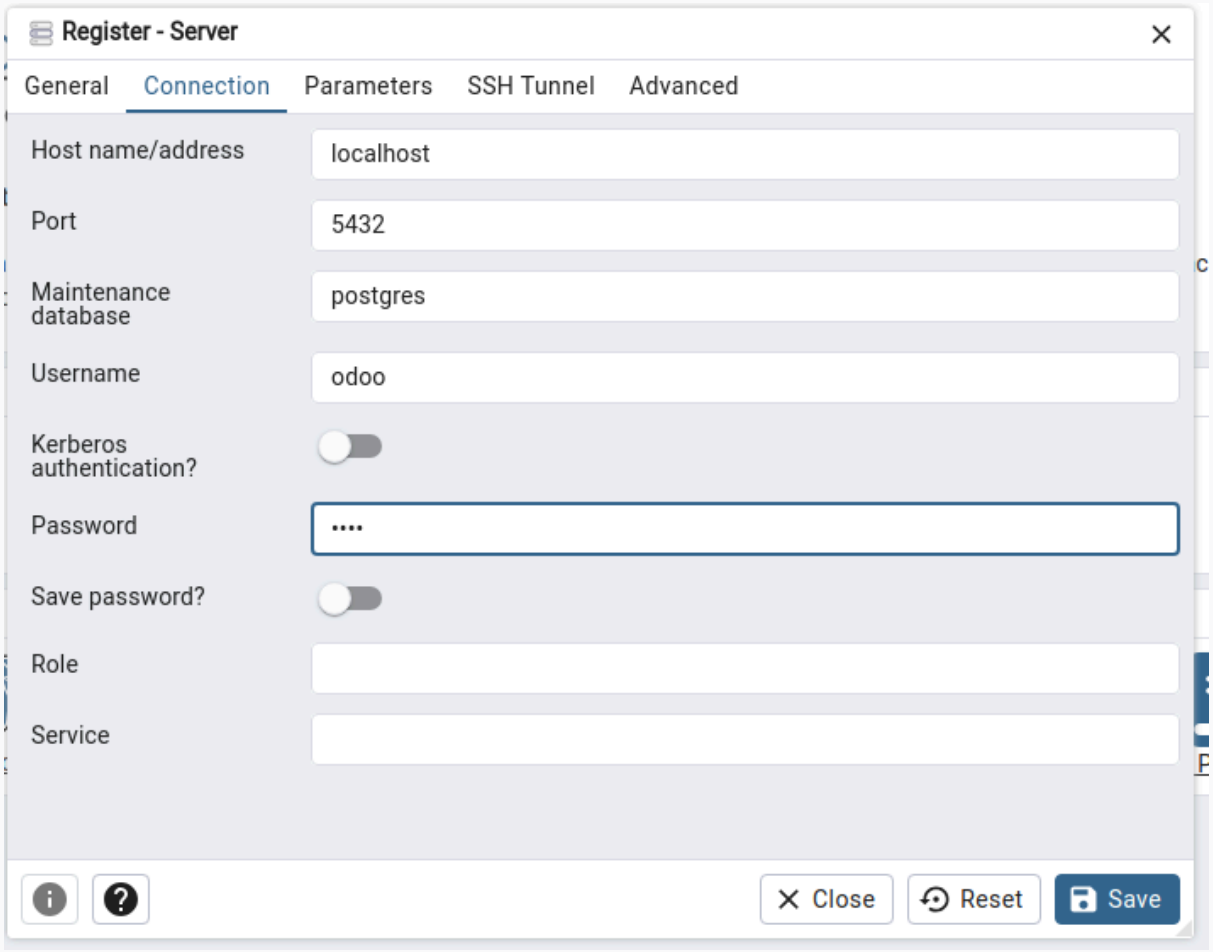
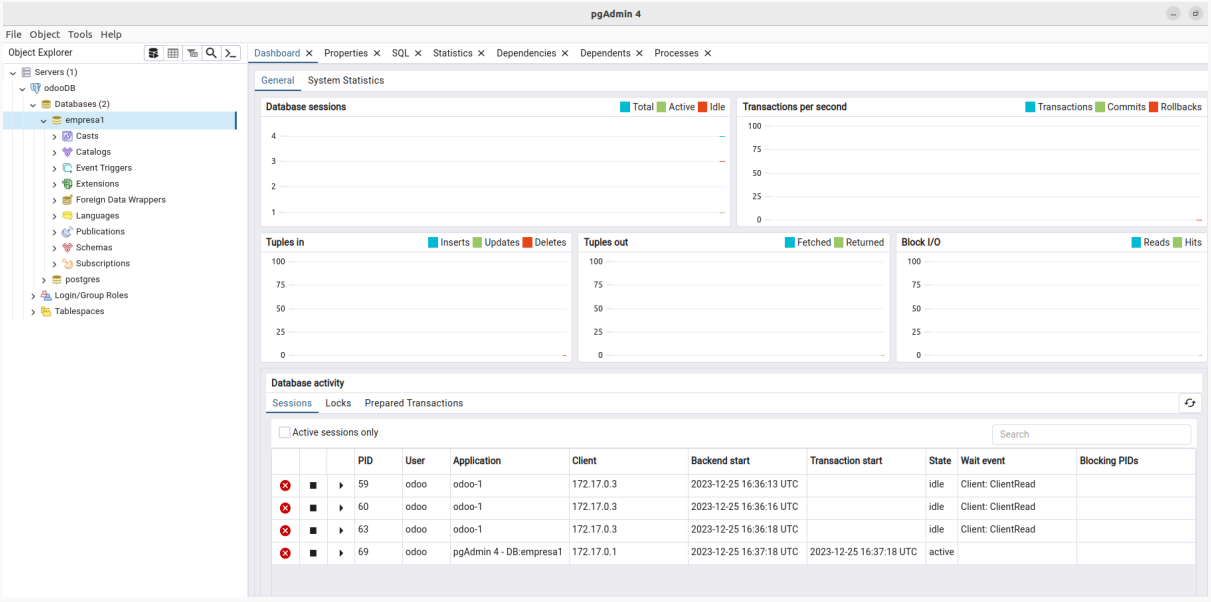


CONFIGURACIÓ

Podem crear un servidor amb l'enllaç "Add New Server":



Necessitarem que PostgresQL sigui accessible (en cas de docker, el mapeig del port 5432 ha d'haver-se fet en crear el contenidor). En cas d'accedir des d'una altra màquina, seria necessari obrir les connexions a PostgresQL, tancades per defecte. Podeu cercar a la documentació de PostgresQL com fer-ho si fos necessari.


Register - Server
General
Connection
Parameters
SSH Tunnel
Advanced
Host name/address
localhost
Port
5432
Maintenance database
postgres
Username
odoo
Kerberos authentication?
Password
Save password?
Role
Service
Close
Reset
Save

Una vegada configurat, podem accedir a la instància de bbdd d'Odoo: